Unsupervised Learning

PTP Program

Preamble

Prerequisite & Objective

To fully understand the material in this session, you are expected to have knowledge about :

1. Python
2. Introduction to Machine Learning
3. Dimensionality reduction

This session is created with the aim for students to :

1. Understand the effect of high dimensional data.
2. Understand the concept of Principal Component Analysis (PCA).
3. Understand the concept of Clustering.
4. Understand matters relating to K-Means (how it works, advantages, limitation, etc).
5. Able to implement K-Means to solve a problem.
6. Understand the concept of K-Modes and K-Prototype.
7. Able to implement K-Prototype to solve a problem.

Dimensionality Reduction

Curse of Dimensionality

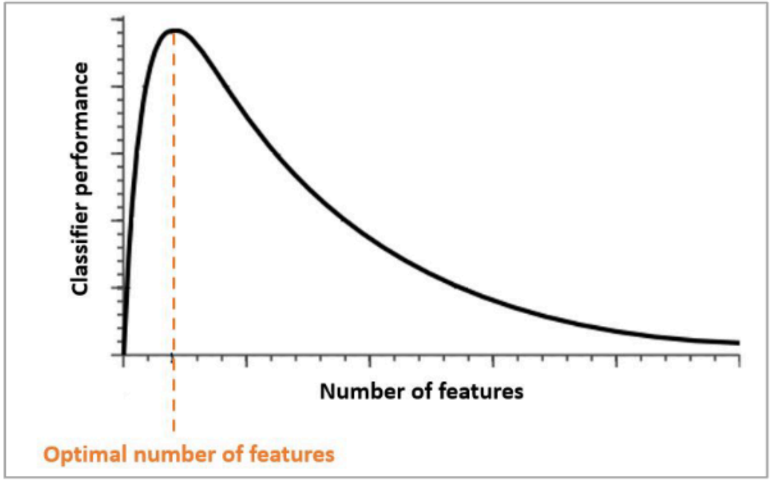
Definition

The curse of dimensionality refers to the difficulties a Machine Learning algorithm faces when working with high dimensional data.

As the number of features (columns) increases, various problems and complexities emerge that can negatively impact the performance and interpretability of models.

An increase in the dimensions can in theory, add more information to the data thereby improving the quality of data but practically increases the noise and redundancy during its analysis.

With the increase in the data dimensions, the model would also increase in complexity and it would become increasingly dependent on the data it is being trained on and can leads to overfitting.



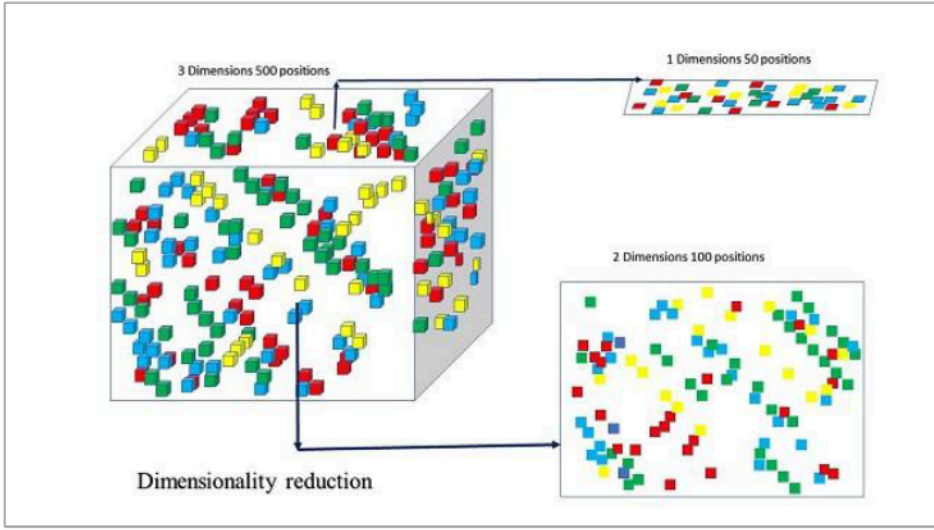
Dimensionality Reduction

Definition

Dimensionality reduction is **a technique to reduce the number of features or dimensions in a dataset while retaining as much relevant information as possible**.

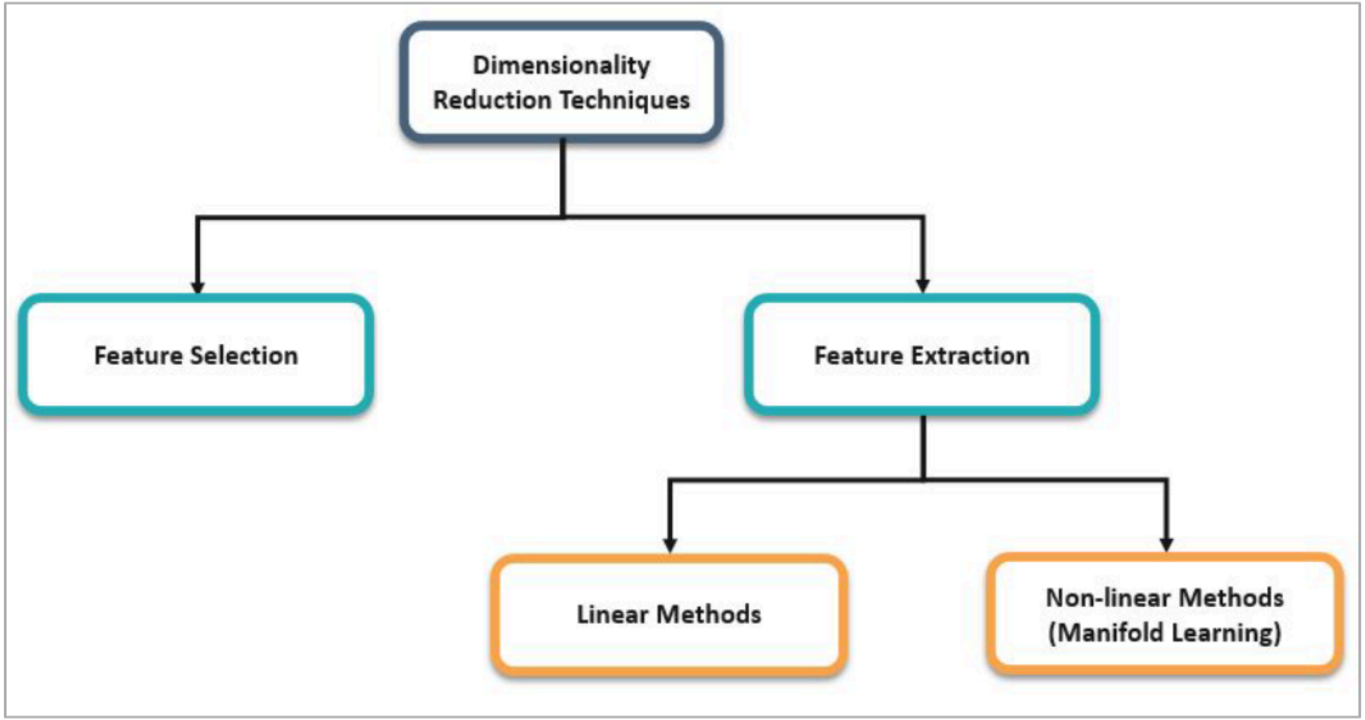
The primary goal of dimensionality reduction is to **simplify complex datasets**, making them more manageable, interpretable, and computationally efficient, while also potentially improving model performance and reducing the risk of overfitting.

Fewer input dimensions often correspond to a simpler model. A model with larger dimensions is more prone to overfitting. So, it is desirable to have more generalized models, and input data with fewer features.



Dimensionality Reduction

Solution



Principal Component Analysis (PCA)

Definition

- Principal Component Analysis (PCA) is a linear transformation process that converts the observations of correlated features into a set of linearly uncorrelated features with the help of orthogonal transformation.
- These new transformed features are called the Principal Components.
- PCA achieves this by finding a set of orthogonal/perpendicular axes (Principal Components) along which the data has the highest variance.
- We can also say that PCA draws straight lines through data like Linear Regression and each straight line represents Principal Components.
- So, in high-dimension data, there are many such straight lines that are possible and the role of PCA is to identify and prioritize them.

Principal Component Analysis (PCA)

Characteristic

- Linearity

  PCA assumes that the relationship between features are linear. It might not perform optimally on data with non linear relationship.
- Variance Maximization

  The first Principal Component captures the most variance, the second captures the second-highest variance, and so on. This ensures that the transformed data retains as much of the original variance as possible.
- Orthogonality

  The Principal Components are orthogonal to each other, meaning they are uncorrelated.
- Dimensionality Reduction

  It helps eliminate less important features while retaining important patterns and relationships.
- Collinearity Handling

  PCA can help address multicollinearity issues by transforming correlated features into uncorrelated Principal Components.
- Continuous Features (Numeric Features)

  It is not recommended to use PCA for categorical features that are encoded.

Principal Component Analysis (PCA)

Steps (1)

Step 1 : Standardization.

Before applying PCA, it's important to standardize the data to have zero mean and unit variance. This ensures that features with larger scales do not dominate the analysis.

Step 2 : Compute the Covariance Matrix.

Calculate the covariance matrix of the standardized data to understand the relationships between features.

Step 3 : Compute Eigenvectors and Eigenvalues.

Solve the eigenvalue-eigenvector problem for the covariance matrix. The eigenvectors represent the principal components, and the eigenvalues represent the amount of variance explained by each component.

Principal Component Analysis (PCA)

Steps (2)

Step 4 : Sort Eigenvectors by Eigenvalues.

Sort the eigenvectors in descending order based on their corresponding eigenvalues. This determines the order of importance of the principal components.

Step 5 : Select Principal Components.

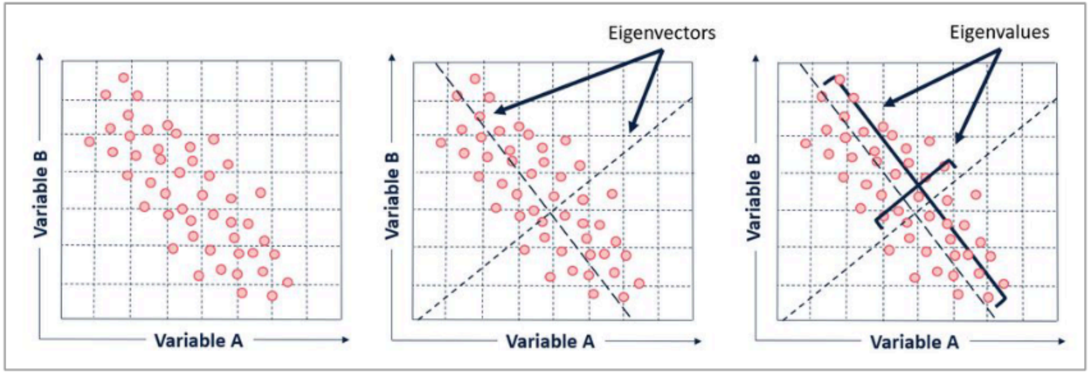
Choose the top k eigenvectors (principal components) that capture the desired amount of variance. These components will form the new lower-dimensional feature space.

Step 6 : Transform Data.

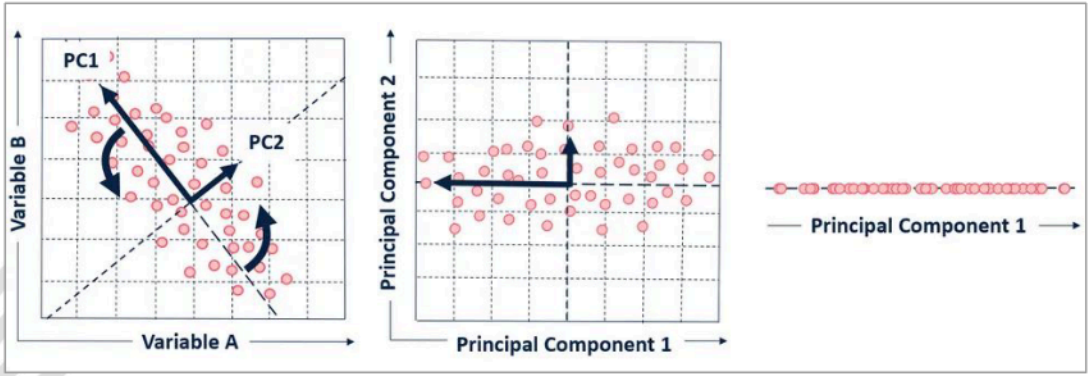
Multiply the original data by the selected principal components to obtain the transformed data in the new coordinate system.

Principal Component Analysis (PCA)

Steps (3)



The example of eigenvectors and eigenvalues of the dataset with two features.



Visual representation of Principal Components PC1 and PC2 that fit on the dataset.

Principal Component Analysis (PCA)

Choosing Number of Components (1)

If you have 8 features, after doing PCA you will have 8 new features. The purpose of doing PCA is to reduce the dimensions so that the number of columns used in this case is no longer 8. The choice of the number of features to be used will be very subjective, allowing for differences between model makers.

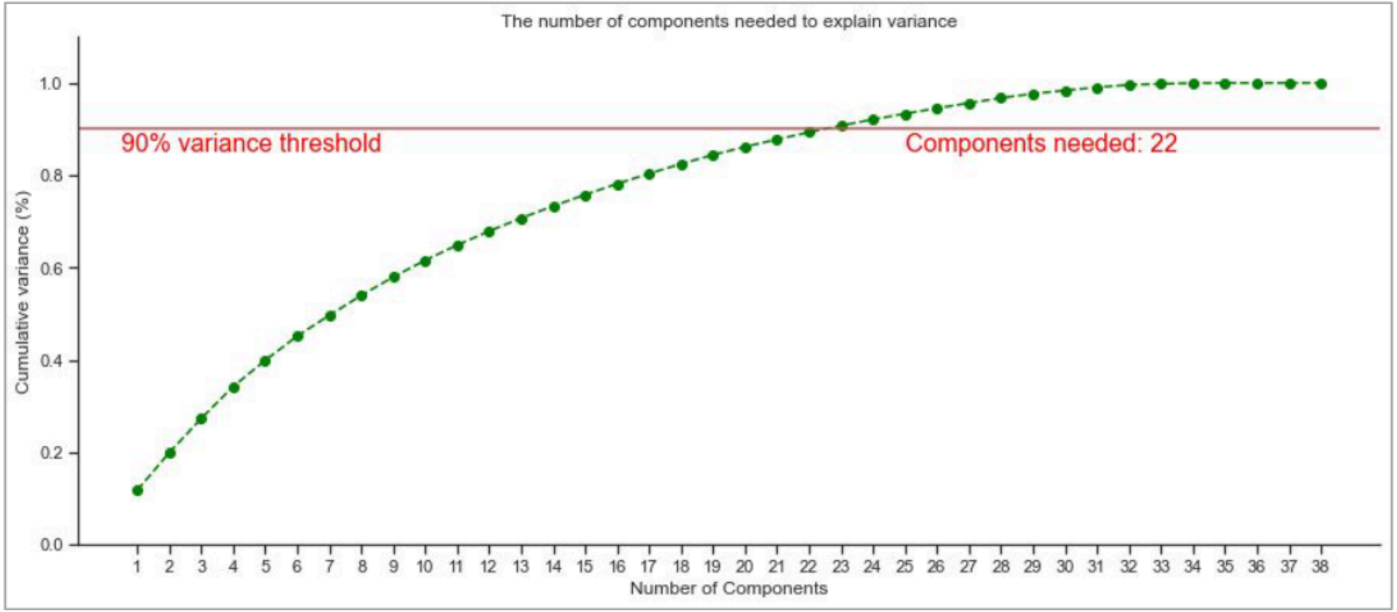
The following are several guidelines that can be used to determine the number of components to be used :

- Preserving information
  How much information you are willing to preserve from the original data. You can choose `n_components=0.9`if you are fine with retaining 90 % of the original information.
- Explained Variance Ratio
  Plot the cumulative explained ratio as a function of the number of components. Choose the number of components where the cumulative explained variance reaches your satisfactory level.
- Scree Plot
  Plot the eigenvalues of the principal components. Select components corresponding to eigenvalues larger than `1`.

Principal Component Analysis (PCA)

Choosing Number of Components (2)

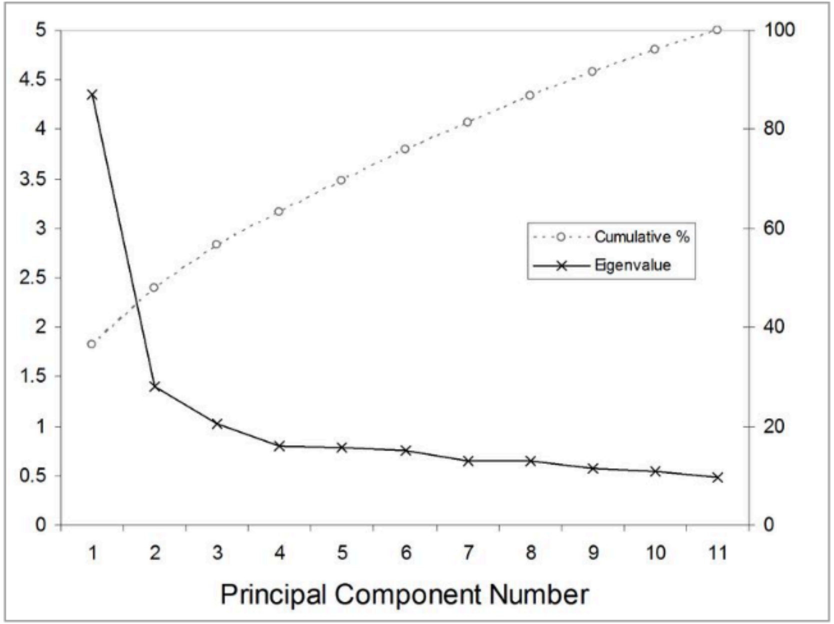
As seen in the image below, if we want to retain 90% of the information from the initial dataset, then the number of features can be reduced to 22.



Principal Component Analysis (PCA)

Choosing Number of Components (3)

In the image below, you can see that eigenvalues greater than 1 are in the number of components 1, 2, and 3. Therefore, the dataset can be reduced to 1-3 features.



Clustering

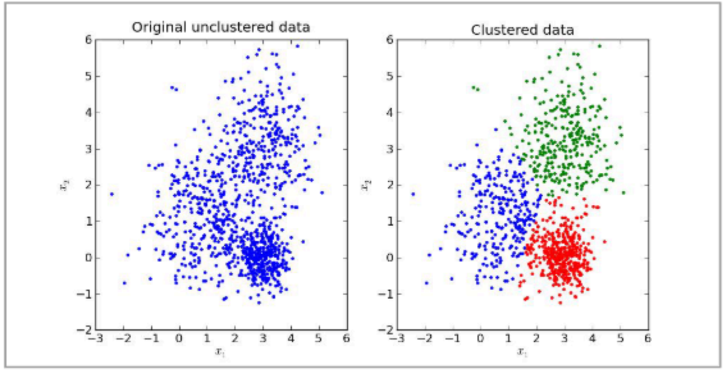
Clustering

Definition

Clustering is the task of dividing data points or samples into a number of groups based on their characteristics such that similar data points fall in the same cluster. Clustering is an Unsupervised Learning technique, so it does not need label.

Some of the most popular applications of clustering :

- ➔ Recommendation system
- ➔ Customer segmentation
- ➔ Image Segmentation
- ➔ Anomaly Detection



Clustering can be divided into 2 types based on clustering results :

- Hard Clustering

  Each data point is assigned to one and only one cluster.
- Soft Clustering

  Each data point can belong to multiple clusters. Each data point will have a probability of being a member of all clusters.

Clustering

Algorithm

Types of clustering algorithm based on how they work :

- Based on centroid
  - → Data points are assigned to the nearest centroid (cluster center) based on a distance metric.
  - → Algorithm example : K-Means
- Based on density
  - → These models search the data space for areas of the varied density of data points in the data space.
  - → They isolate different dense regions and assign the data points within these regions to the same cluster.
  - → Algorithm example : DBSCAN (Density-Based Spatial Clustering Applications with Noise).
- Based on distribution
  - → Assumes that data points are generated from a mixture of probability distributions (example: Gaussian Distribution).
  - → Algorithm example : Gaussian Mixture Models (GMM).
- Based on hierarchy
  - → Organizes data points into a hierarchy of nested clusters (dendrogram)
  - → Algorithm example : Agglomerative Hierarchical Clustering..

K-Means

Definition

K-Means is a centroid-based clustering algorithm that aims to partition a dataset into `K` distinct, non-overlapping clusters (hard clustering).

Key characteristics of K-Means:

- In each cluster, a centroid will be formed. Centroid is the center point of a cluster. The goal is to minimize the distance between data points and their respective cluster centroids.

- It divides the dataset into K clusters, where each data point belongs to the cluster with the nearest centroid. K-Means is hard clustering algorithm which means each data point will belong to one cluster only.

- K-Means uses a distance metric distance metric to measure the dissimilarity between data points and centroids. Euclidean Distance is usually used as a distance metric for K-Means.

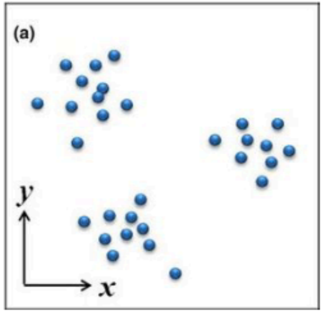
K-Means

Steps (1)

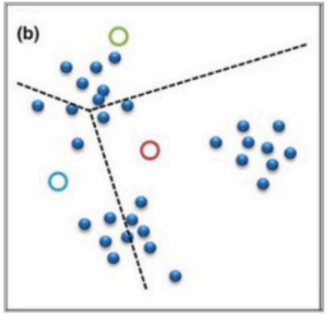
The following are the steps that KMeans takes if a dataset like the image on the side wants to be clustered into 3 clusters.

Step 1 : Initialization.

- ➔ Start by randomly selecting K initial centroids.
- ➔ These can be chosen from the dataset itself or by other methods like random initialization.



Original dataset



Step 1 : Initialization

K-Means

Steps (2)

Step 2 : Assignment.

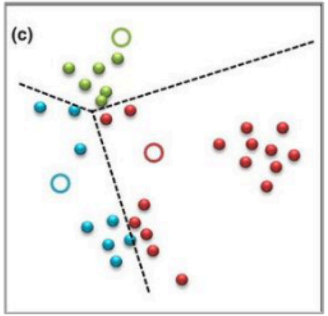
- ➔ Assign each data point to the cluster represented by the nearest centroid.
- ➔ This assignment is based on the Euclidean distance or another chosen distance metric.

Step 3 : Update centroids.

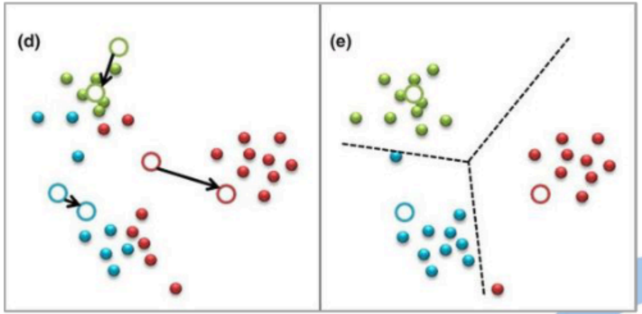
- ➔ Recalculate the centroids for each cluster as the mean of all data points currently assigned to that cluster.

Step 4 : Repeat step 2 and 3.

- ➔ Repeat the assignment and centroid update steps iteratively until one of the stopping criteria is met.
- ➔ Common stopping criteria include a fixed number of iterations, convergence of centroids, or a predefined threshold.



Step 2 : Assignment



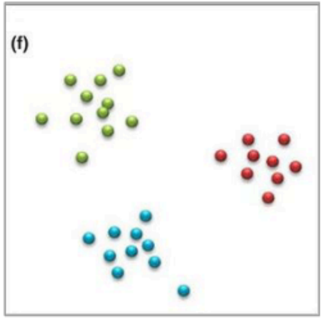
Step 3 : Update centroids

K-Means

Steps (3)

Step 5 : Final clustering.

→ After convergence, the algorithm produces K clusters, where each data point is assigned to one of these clusters based on the final centroids.



Step 5 : Final clustering

K-Means

Advantages & Limitations

Advantages

1. Simple and suitable for large dataset.
2. Guaranteed to converge in a finite number of steps (usually quite small iterations).
3. One of the fastest clustering algorithms.

Limitations

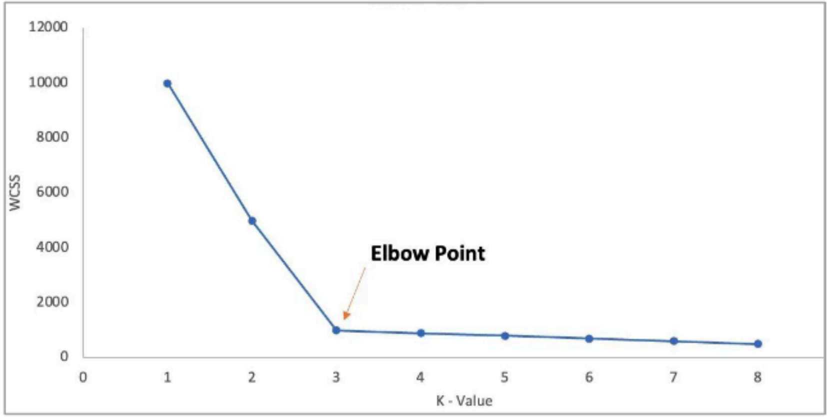
1. Sensitive to the initial place of centroids, which can lead to different results for different initializations.
2. Assumes cluster are spherical, equally sized, and have similar densities.
3. The number of clusters (`K`) needs to be specified in the beginning before clustering start.
4. Sensitive to outliers.
5. K-Means is designed for continuous features. It may not work well for categorical features that have undergone feature encoding.

K-Means

Choose the Number of Clusters (1)

There are several methods and techniques to help determine an optimal cluster :

1. Elbow method
   - ➔ In order to choose with elbow method, first, plot the Within-Cluster Sum of Squares (WCSS). WCSS is the cost function against different values of K.
   - ➔ WCSS measures the total variance within each cluster. It decreases as K increases because smaller clusters can fit the data points more closely.
   - ➔ Look for an "elbow point" on the plot where the rate of decrease in WCSS starts to slow down. This point indicates a potential optimal K.
   - ➔ Keep in mind that **the elbow method is not always definitive, and the choice of K can be subjective**.



K-Means

Choose the Number of Clusters (2)

2. Silhouette score
    - ➔ The silhouette score measures how similar each data point is to its own cluster compared to other clusters.
    - ➔ Calculate the silhouette score for different values of K.
    - ➔ Higher silhouette scores indicate clear clusters.
    - ➔ **Choose the K that maximizes the silhouette score.**
    - ➔ Range of silhouette score between -1 to 1.

      `1` : Points are perfectly assigned in a cluster. There is no overlapping between clusters.

      `0` : Cluster are overlapping.

      `-1` : Points might have got assigned to the wrong clusters.

3. Silhouette Plot:
    - ➔ Create silhouette plots for different values of `K`.
    - ➔ Silhouette plots display the silhouette score for each data point within its cluster.
    - ➔ Inspect the plots to identify `K` with the highest average silhouette score and well-differentiated clusters.

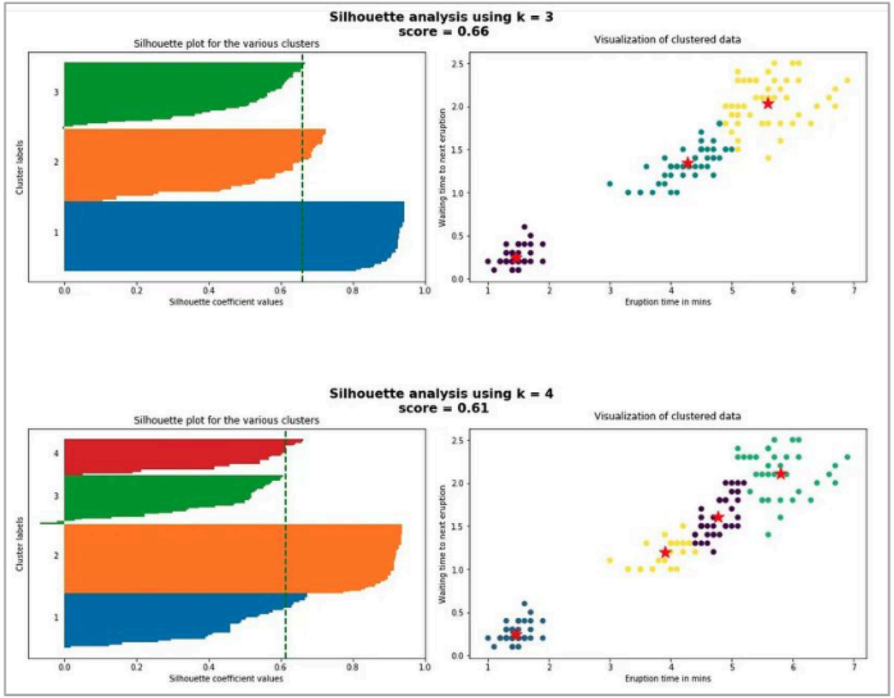
K-Means

Choose the Number of Clusters (3)

The following conditions should be checked to pick the right cluster using Silhouette Plot :

- All the cluster should have a silhouette score **more than average score** of the dataset (green line in image beside).
- There **should not be wide fluctuations in the size** of the clusters.

So, based on the Silhouette Plot beside, `cluster=3` is better because all cluster beyond the green line and the size of each cluster is similar.



K-Means

Choose the Number of Clusters (4)

4. Domain Knowledge:
   - ➔ Sometimes, domain knowledge can provide insights into the appropriate number of clusters. For example, in customer segmentation, there may be a predefined number of customer segments.

5. Visual Inspection:
   - ➔ Visualize the clustering results for different values of K and inspect whether the clusters make sense and align with your objectives.

Summary : the choice of `K` **can be somewhat subjective** and may depend on the specific problem and goals. It's often a good practice to **try multiple methods and validate the chosen `K`** by assessing the quality of the resulting clusters in a real-world context.

K-Modes

Definition

K-Modes is a clustering algorithm similar to K-Means, but it is **designed to work with categorical data**. While K-Means uses means to represent cluster centroid, K-Modes uses modes, which are the most frequent category values within each cluster.

The following are the steps of K-Modes :

Step 1 : Initialization.

- ➔ Start by randomly selecting K initial centroids.
- ➔ These centroids are represented by the mode of each categorical feature within the cluster.
- ➔ You can use random initialization or more advance techniques.

Step 2 : Assignment.

- ➔ Assign each data point to the cluster whose mode (most frequent category values) is most similar to the data point's categorical values.
- ➔ The similarity measure used is often a dissimilarity metric such as the Hamming distance.

K-Modes

Definition

Step 3 : Update modes.

- ➔ After assigning data points to clusters, update the modes of each cluster by selecting the most frequent category values within that cluster.

Step 4 : Repeat.

- ➔ Iteratively repeat the assignment and mode update steps until convergence.
- ➔ Convergence occurs when the assignment of data points to clusters no longer changes significantly or when a maximum number of iterations is reached.

Step 5 : Final clustering.

- ➔ The algorithm produces `K` clusters, each represented by a mode vector containing the most frequent category values for each categorical feature within the cluster.

K-Prototype

Definition

K-Prototype is a clustering technique that **used to cluster data with a combination of numerical and categorical features**.
K-Prototype combines the technique of K-Means and K-Modes. The steps performed in K-Prototype are exactly the same as the steps performed in K-Means and K-Modes.

External References

Colab Link 1 —— *Visit Here*

Colab Link 2 —— *Visit Here*